

学号： 2104230414

沈阳建筑大学

上机报告

课程名称： 算法设计与分析

项目名称： 分支限界法

2023年秋季学期

上机时间：第 7 周 星期 二 第 3 ~ 4 节

学生姓名： 张清晨 专业班级： 计算机 2101

指导教师： 刘俊岭 成 绩：

项目名称	分支限界法		学时	4
日期：2023 年 10 月 10 日		第 7 教学周		
硬件及软件				
名 称		版本、型号		
Visual C++		6.0 及以上		
上机目的与要求： 1. 问题描述，题目定义。 2. 算法实例，实例演示程序过程。 3. 程序代码，完整可运行程序，并加注释。 4. 程序验证，要求有不同的输入、输出，并截图。 5. 算法分析，时间复杂度。 本次任务： 实验 5-5：图的 m 着色问题(回溯法) 实验 6-1：装载问题(分支限界法) 实验 6-2：旅行商问题(分支限界法)				

实验 5-5: 图的 m 着色问题(回溯法)

1. 问题描述, 题目定义。

给定无向连通图 $G=(V, E)$ 和正整数 m , 求最小的整数 m , 使得用 m 种颜色对 G 中的顶点着色, 使得任意两个相邻顶点着色不同。

2. 算法实例, 实例演示程序过程。

示例 1: 结点数 4 边数 2

输入:

4

2

0 1 1 0

1 0 0 1

1 0 0 1

0 1 1 0

输出:

要求:

- 1 填空完成程序;
- 2 输出最少旅行费用。
- *3 报告中指出是程序使用是队列还是优先队列。

 E:\算法设计与分析\6\图的m着色问题(回溯法).exe

输入顶点数 n 和着色数 m :

4 2

输入无向图的邻接矩阵:

0 1 1 0

1 0 0 1

1 0 0 1

0 1 1 0

着色所有可能的解:

1 2 2 1

2 1 1 2

一共有2种着色方案

请按任意键继续. . .

3. 程序代码，完整可运行程序，并加注释。

```
/**
回溯法-m 着色
输入顶点数 n 和着色数 m:
4 3

输入无向图的邻接矩阵:
0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0
*/

#define _CRT_SECURE_NO_WARNINGS
#include<iostream>
#include<stdio.h>
int color[100];//存储各个顶点的颜色
int cnt = 0;//存储方案数
bool Check(int k, int c[100][100]);
void GraphColor(int n, int m, int c[100][100]);

int main(){
    int i, j, n, m;
    int c[100][100];//存储 n 个顶点的无向图的数组
    printf("输入顶点数 n 和着色数 m:\n");
    scanf("%d%d", &n, &m);

    printf("\n");
    printf("输入无向图的邻接矩阵:\n");
    for (i = 1; i <= n; i++){
        for (j = 1; j <= n; j++){
            scanf("%d", &c[i][j]);
        }
    }

    printf("\n");
    printf("着色所有可能的解:\n");
    GraphColor(n, m, c);

    if (cnt == 0){
        printf("NO\n");
    }
    else{
        printf("一共有%d 种着色方案\n", cnt);
    }
}
```

```

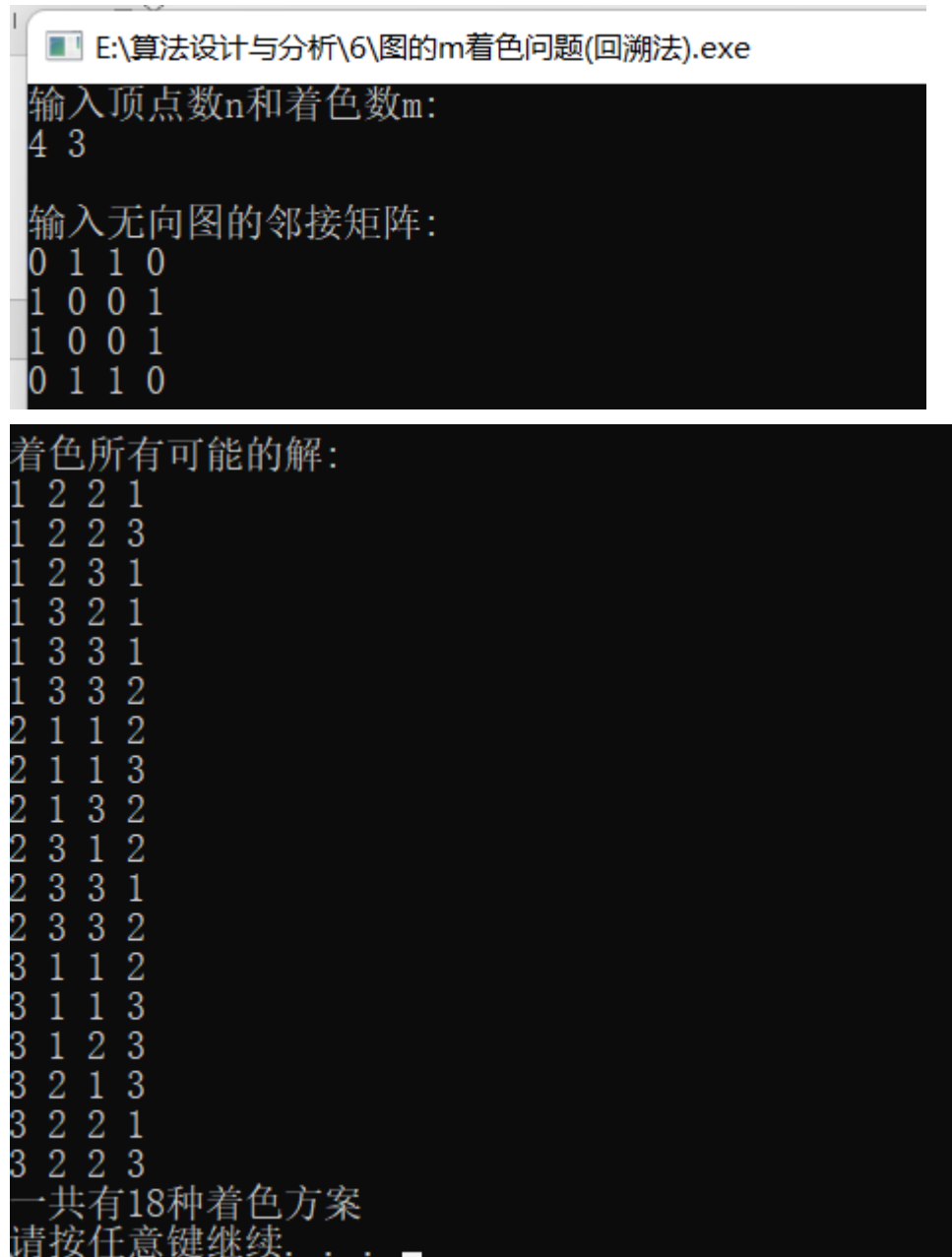
    system("PAUSE");
    return 0;
}

bool Check(int k, int c[100][100]){//判断顶点 k 的着色是否发生冲突
    int i;
    //G 的每条边的两个顶点着不同颜色
    for (i = 1; i<k; i++){
        if (c[k][i] == 1 && color[i] == color[k]){
            return false;
        }
    }
    return true;
}

void GraphColor(int n, int m, int c[100][100]){
    int i, k;
    for (i = 1; i <= n; i++){
        color[i] = 0;
    }
    k = 1;
    while (k >= 1){
        color[k] = color[k] + 1;
        while (color[k] <= m){
            if (Check(k, c)){
                break;
            }
            else{
                color[k] = color[k] + 1;//搜索下一个颜色
            }
        }
        if (color[k] <= m && k == n){//输出
            for (i = 1; i <= n; i++){
                printf("%d ", color[i]);
            }
            printf("\n");
            cnt++;
        }
        else if (color[k] <= m && k < n){
            k = k + 1;//处理下一个顶点
        }
        else{
            color[k] = 0;
            k = k - 1;//回溯
        }
    }
}
}

```

4. 程序验证，要求有不同的输入、输出，并截图。



```
E:\算法设计与分析\6\图的m着色问题(回溯法).exe
输入顶点数n和着色数m:
4 3

输入无向图的邻接矩阵:
0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0

着色所有可能的解:
1 2 2 1
1 2 2 3
1 2 3 1
1 3 2 1
1 3 3 1
1 3 3 2
2 1 1 2
2 1 1 3
2 1 3 2
2 3 1 2
2 3 3 1
2 3 3 2
3 1 1 2
3 1 1 3
3 1 2 3
3 2 1 3
3 2 2 1
3 2 2 3
一共有18种着色方案
请按任意键继续...
```

5. 算法分析，时间复杂度。

该问题使用了回溯算法。在回溯的过程中进行了部分剪枝，主要是判断顶点的着色是否发生冲突，利用 `check` 函数中的 `c[k][i] == 1 && color[i] == color[k]` 来判断，如果一样，则返回 `false`，否则返回 `true`，对算法的可行性解空间进行了缩小，使得搜索更加高效。本题目如果不考虑剪枝，其时间复杂度为 $O(m^n)$, m 为着色数， n 为顶点数。使用了长度为 n 的二维数组，所以空间复杂度为 $n*n$ 。

时间复杂度: $O(m^n)$

空间复杂度: $O(n*n)$

实验 6-1: 装载问题(分支限界法)

1. 问题描述, 题目定义。

有一批共 n 个集装箱要装上两艘载重量分别为 c_1 和 c_2 的轮船, 其中集装箱 i 的重量为 w_i , 且 $w_1 + w_2 + \cdots + w_n \leq c_1 + c_2$ 。装载问题要求确定是否有一个合理的装载方案可将这些集装箱装上这两艘轮船。如果有, 找出一种装载方案。

2. 算法实例, 实例演示程序过程。

示例 1:

当 $n=3$, $c_1=c_2=50$, $w=\{10, 40, 40\}$ 时, 则可以将集装箱 1 和 2 装到第一艘轮船上, 而将集装箱 3 装到第二艘轮船上。如果 $w=\{20, 40, 40\}$, 则无法将这 3 个集装箱都装上轮船。

要求:

- 1 填空完成程序;
- 2 输出所选物品重量及价值, 总重量与价值。
- *3 报告中指出是队列式还是优先队列式。

E:\算法设计与分析\6\装载问题(分支限界法).exe

```
第一艘船装载的方案为: 0 0 1
第一艘船装载情况为: 装第3个集装箱(该物品重量为40)
第一艘船的装载总重量: 40
请按任意键继续. . .
```

3. 程序代码, 完整可运行程序, 并加注释。

```
/*exp6-1 maxLoadingBB MaxLoading,分支限界法*/
```

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include<queue>

using namespace std;

template<class Type>
class QNode
{
    template<class T>
    friend Type MaxLoading(Type*, Type, int, int*);
    template<class U>
```

```

    friend void EnQueue(queue<QNode<Type>*>& Q, Type wt, int i, int n, Type bestw,
QNode<Type>* E, QNode<Type>* &bestE, int* bestx, bool ch);
public:
    QNode* parent;
    bool LChind;
    Type weight;
};

template<class Type>
void EnQueue(queue<QNode<Type>*>& Q, Type wt, int i, int n, Type bestw,
QNode<Type>* E, QNode<Type>* &bestE, int* bestx, bool ch)
{
    if (i == n)
    {
        if (wt == bestw)
        {
            bestE = E;
            bestx[n] = ch;
        }
        return;
    }
    QNode<Type>* b;
    b = new QNode<Type>;
    b->weight = wt;
    b->parent = E;
    b->LChind = ch;
    Q.push(b);
}

template<class Type>
Type MaxLoading(Type* w, Type c, int n, int* bestx)
{
    queue<QNode<Type>*> Q;
    Q.push(nullptr);
    int i = 1;
    Type cw = 0,
        bestw = 0,
        r = 0;
    for (int i = 2; i <= n; i++)
        r += w[i];
    QNode<Type>* Ew = nullptr, //Ew 这里作为一个中间量，连接 parent 和 child，赋 0
是因为树的根的值是 0，while 刚开始的时候其代表 root
        *bestE = nullptr; //bestE 的作用是：结束 while 循环后，bestE 指向最优解的叶
子节点，然后通过 bestE->parent 找到装入了哪些物品
    while (true)
    {
        Type wt = cw + w[i];

```



```

        if (wt < c) //首先检测当前扩展结点的左儿子结点是否为可行结点。如果是，
        则将其加入到活结点队列 Q 中
        {
            if (wt > bestw) // 可行结点
                bestw = wt; //提前更新 bestW,注意更新条件
            EnQueue(Q, wt, i, n, bestw, Ew, bestE, bestx, true);// 加入活结点队列
        }
        if (cw + r >= bestw)//将其右儿子结点加入到活结点队列中(右儿子结点一定是
        可行结点)。
            EnQueue(Q, cw, i, n, bestw, Ew, bestE, bestx, false);
            Ew = Q.front();//活结点队列中，队首元素被取出作为当前扩展结点。
            Q.pop();// 取下一扩展结点
            if (!Ew) //如果取得的数是 0，代表该处理下一层
            {
                if (Q.empty())//活结点队列已空，算法终止。
                    break;
                Q.push(nullptr); //如果队列中还有数据，表示循环还没结束。在该层的末
                尾加一个 nullptr 标识符,相当于讲稿中-1
                Ew = Q.front();
                Q.pop();
                i++;
                r -= w[i];//计算剩余的重量
            }
            cw = Ew->weight;//更新最新节点的值
        }
        for (int j = n - 1; j > 0; j--)
        {
            bestx[j] = bestE->LChind;
            bestE = bestE->parent;
        }

        cout << "第一艘船装载的方案为: ";
        for (int i = 1; i <= n; i++)
            cout << bestx[i] << " ";
        cout << endl;
        cout << "第一艘船装载情况为: ";
        for (int i = 1; i <= n; i++)
            if (bestx[i] == 1)
            {
                cout << "装第" << i << "个集装箱" << "(该物品重量为" << w[i] << ")" << endl;
            }

        return bestw;
    }
}

```

```

int main()

```

```

{
    int w[4] = { 0, 20, 40, 40 };
    int c = 50;
    int n = 3;
    int* bestx = new int[4];

    cout << "第一艘船的装载总重量: " << MaxLoading<int>(w, c, n, bestx) << endl;

    system("PAUSE");
    return 0;
}

```

4. 程序验证，要求有不同的输入、输出，并截图。

```

int w[4] = { 0, 10, 40, 40 };
int c = 50;
int n = 3;
int* bestx = new int[4];

```

E:\算法设计与分析\6\装载问题(分支限界法).exe

```

第一艘船装载的方案为: 1 0 1
第一艘船装载情况为: 装第1个集装箱(该物品重量为10)
装第3个集装箱(该物品重量为40)
第一艘船的装载总重量: 50
请按任意键继续. . .

```

```

int w[4] = { 0, 20, 40, 40 };
int c = 100;
int n = 3;
int* bestx = new int[4];

```

```
第一艘船装载的方案为: 1 1 1
第一艘船装载情况为: 装第1个集装箱(该物品重量为20)
装第2个集装箱(该物品重量为40)
装第3个集装箱(该物品重量为40)
第一艘船的装载总重量: 100
请按任意键继续. . .
```

5. 算法分析，时间复杂度。

本题使用分支界限法求解装载问题，相比较于回溯法，回溯法的求解目标是找出解空间中满足约束条件的所有解，而分支界限法的求解目标则是找出满足约束条件的一个解，或是满足约束条件的解中找出使某一目标函数值达到极大或极小的解，即在某种意义下的最优解。另外还有一个非常大的不同点就是，回溯法以深度优先的方式搜索解空间，而分支界限法则以广度优先的方式或以最小耗费优先的方式搜索解空间。其时间复杂度为 $O(2^n)$ 。空间复杂度，本次算法用了队列、链表等数据结构，构造了一棵二叉树，所以空间复杂度为 $O(n)$ 。所以

- 时间复杂度: $O(2^n)$
- 空间复杂度: $O(2^n)$

实验 6-2: 旅行商问题(分支限界法)

1. 问题描述，题目定义。

售货员要到 n 个城市去推销商品，已知各城市之间的路程,请问他应该如何选定一条从城市 1 出发，经过每个城市一遍，最后回到城市 1 的路线，使得总的周游路程最小。

2. 算法实例，实例演示程序过程。

示例 1: 结点数 4 边数 6

输入:

4

6

1 2 30

1 3 6

1 4 4

2 3 5

2 4 10

3 4 20

输出:

要求:

- 1 填空完成程序;
- 2 输出最少旅行费用。
- *3 报告中指出是程序使用是队列还是优先队列。

```
6A算法设计与分析(实验6-2): 旅行商问题(分支限界法).exe
最小出边和(lcost): 14 子树费用的下界(lcost): 20 当前费用(cc): 6 节点所在层(s): 1
当前解是(x): 1-3-2-4
--- 当前优先队列: D(20) --- 当前最优值 (bestC): -1
===== Node: E =====
最小出边和(lcost): 14 子树费用的下界(lcost): 18 当前费用(cc): 4 节点所在层(s): 1
当前解是(x): 1-4-3-2
--- 当前优先队列: E(18)-D(20) --- 当前最优值 (bestC): -1
***** 开始一个新节点扩展 *****
===== Node: F =====
最小出边和(lcost): 10 子树费用的下界(lcost): 34 当前费用(cc): 24 节点所在层(s): 2
当前解是(x): 1-3-2-4
--- 当前优先队列: D(20)-F(34) --- 当前最优值 (bestC): -1
===== Node: G =====
最小出边和(lcost): 10 子树费用的下界(lcost): 24 当前费用(cc): 14 节点所在层(s): 2
当前解是(x): 1-3-2-3
--- 当前优先队列: D(20)-G(24) --- 当前最优值 (bestC): -1
***** 开始一个新节点扩展 *****
===== Node: H =====
最小出边和(lcost): 9 子树费用的下界(lcost): 20 当前费用(cc): 11 节点所在层(s): 2
当前解是(x): 1-3-2-4
--- 当前优先队列: H(20)-G(24) --- 当前最优值 (bestC): -1
===== Node: I =====
最小出边和(lcost): 9 子树费用的下界(lcost): 35 当前费用(cc): 26 节点所在层(s): 2
当前解是(x): 1-3-4-2
--- 当前优先队列: H(20)-G(24)-F(34) --- 当前最优值 (bestC): -1
||||| 到达叶子节点的父节点 --- 并更新最优解 |||||
===== Node: H =====
最小出边和(lcost): 9 子树费用的下界(lcost): 20 当前费用(cc): 11 节点所在层(s): 2
当前解是(x): 1-3-2-4
--- 当前优先队列: G(24)-F(34) --- 当前最优值 (bestC): -1
||||| 到达叶子节点的父节点 --- 不更新 |||||
===== Node: G =====
最小出边和(lcost): 10 子树费用的下界(lcost): 24 当前费用(cc): 14 节点所在层(s): 2
当前解是(x): 1-4-2-3
--- 当前优先队列: H(25)-F(34) --- 当前最优值 (bestC): 25
=====
最短路径为: 25
1 -> 3 -> 2 -> 4 -> 1 请按任意键继续. . .
Process exited after 0.7316 seconds with return value 0
请按任意键继续. . .
```

3. 程序代码，完整可运行程序，并加注释。

```
/**
//分支限界法-旅行商(TSP)问题
输入
4
6
1 2 30
1 3 6
1 4 4
2 3 5
2 4 10
3 4 20
*/

#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <vector>
#include <cstring>
#include <iomanip>
#include <queue>
#define NoEdge -1
#define NN 50 // 可执行的最大的顶点个数
using namespace std;
int n; // 图的顶点个数
int adjMatrix[NN][NN]; // 图的邻接矩阵
int v[50]; // 最优解
int bestC; // 最优值
int num = 0; // 节点序号
```

```

/*****
* 函数描述：  数据输入以及内存的初始化
*****/
void input()
{
    cin >> n; // 输入顶点个数
    int k;
    memset(adjMatrix, NoEdge, sizeof(adjMatrix)); // 邻接矩阵的内存初始化
    cin >> k; // 输入边的个数;
    int p, q, len;
    // 初始化邻接矩阵
    for (int i = 1; i <= k; ++i)
    {
        cin >> p >> q >> len;
        adjMatrix[p][q] = len;
        adjMatrix[q][p] = len;
    }
}

/*****
* 函数描述：  格式化打印结果
* 参数描述：  res，最优值
*****/
void printTravel(int res)
{
    if (res == NoEdge)
        cout << "===== 无法形成回路 =====" << endl;
    else
    {
        cout << "\n===== \n 最短路径为: " << res
        << endl;
        for (int i = 1; i <= n; i++)
            cout << v[i] << " ---> ";
        cout << v[1];
    }
}

/*****
* 类描述： 最小堆（队列中元素类型）
* 参数描述：
x，用于记录当前解；
s，表示节点在排列树中的层次，从排列树的根节点到该节点的路径为 x[0:s]，
需要进一步搜索的顶点是 x[s+1:n-1]。
cc，表示当前费用，
rcost，是子树费用的下界，
lcost，是 x[x:n-1]中顶点最小出边费用和。
*****/

```

```

class MinHeapNode
{
public:
    char name; // 节点的序号
    int lcost, // x[s:n-1]中顶点最小出边费用和
        rcost, // 子树费用的下界
        cc;    // 当前费用
    int s,     // 根节点到当前节点的路径为 x[0:s]
        *x;    // 需要进一步搜索的顶点是 x[s+1:n-1]

    // 构造节点并递增序号
    MinHeapNode()
    {
        num += 1;
        name = num + 'A';
    }

    // 最小堆中使用下界排序
    bool operator<(const MinHeapNode &MH) const
    {
        return rcost > MH.rcost;
    }

    // 打印节点信息
    void printNode(priority_queue<MinHeapNode> pq)
    {
        cout << "===== Node: " << name << " =====" << endl;
        cout << "最小出边和(lcost): " << lcost << "\t 子树费用的下界(lcost): " << rcost
            << "\t 当前费用(cc): " << cc << " \t 节点所在层(s): " << s << endl;
        cout << "当前解是(x): ";
        for (int i = 0; i < n - 1; ++i)
            cout << x[i] << "-";
        cout << x[n - 1] << endl;
        // 输出优先级队列
        if (!pq.empty())
        {
            cout << "-- 当前优先队列: ";
            for (int i = 0; i < pq.size(); ++i)
            {
                cout << pq.top().name << "(" << pq.top().rcost << ")-";
                pq.pop();
            }
        }
        else
        {
            cout << "(优先级队列为空)" << endl;
        }
    }
}

```

```

        cout << "-- 当前最优值 (bestC): " << bestC << endl;
    }
};

/*****
* 算法描述：核心算法
算法开始时创建一个最小堆，表示活节点优先队列。堆中每个节点的 lcost
值是优先队列的优先级。接着计算出图中每个顶点的最小费用出边并用 Minout 记录。
如果所给的有向图中某个顶点没有出边，则该图不可能有回路，算法即告结束。
如果每个顶点都有出边，则根据计算出的 Minout 作算法初始化。
*****/
int BBTSP()
{
    priority_queue<MinHeapNode> pq; // 优先级队列
    MinHeapNode E;                // 最小堆节点

    int cc, lcost, MinSum, *MinOut, lb;
    int i, j;

    MinSum = 0;                    // 最小出边费用和
    MinOut = new int[n + 1]; // 计算 MinOut[i] = 顶点 i 的最小出边费用
    for (i = 1; i <= n; i++)
    {
        MinOut[i] = NoEdge; // 所有的出边初始化为无连接
        // 遍历找出 MinOut[i] = 顶点 i 的最小出边费用
        for (j = 1; j <= n; j++)
            if (adjMatrix[i][j] != NoEdge && (adjMatrix[i][j] < MinOut[i] || MinOut[i] ==
NoEdge))
                MinOut[i] = adjMatrix[i][j];
        // 不存在与这个顶点相连接的边
        if (MinOut[i] == NoEdge)
            return NoEdge;
        MinSum += MinOut[i];
    }

    // 初始化最小堆
    E.s = 0;                // 根节点到当前节点的路径为 x[0:s]
    E.cc = 0;                // 当前费用为 0
    E.lcost = MinSum; // x[s:n-1]中顶点最小出边费用和
    E.x = new int[n]; // 需要进一步搜索的顶点是 x[s+1:n-1]
    // 初始化为顺序搜索
    for (i = 0; i < n; i++)
        E.x[i] = i + 1;
    bestC = NoEdge; // 初始化最优值为 NoEdge

    E.printNode(pq);
    //搜索排列空间树

```

```

while (E.s < n - 1) //非叶节点
{
    if (E.s == n - 2) // 当前扩展节点是叶节点的父节点, 判断构成的回路是否最优
    {
        if (adjMatrix[E.x[n - 2]][E.x[n - 1]] != NoEdge && adjMatrix[E.x[n - 1]][1] != NoEdge &&
            (E.cc + adjMatrix[E.x[n - 2]][E.x[n - 1]] + adjMatrix[E.x[n - 1]][1] <
bestC || bestC == NoEdge))
        { // 如果更优, 则更新费用更小的路
            cout << "\n|||||||||||||||| 到达叶子节点的父节点 ---- 并更新最优解
||||||||||||||||"
                << endl;
            E.printNode(pq);
            bestC = E.cc + adjMatrix[E.x[n - 2]][E.x[n - 1]] + adjMatrix[E.x[n - 1]][1];

            E.cc = bestC;
            E.rcost = bestC;
            E.s++;
            pq.push(E);
        }
        else
        {
            cout << "\n|||||||||||||||| 到达叶子节点的父节点 ---- 不更新
||||||||||||||||"
                << endl;
            E.printNode(pq);
            delete[] E.x; // 舍弃需要进一步搜索的节点
        }
    }
    else // 产生当前扩展节点儿子节点
    {
        cout << "\n***** 开始一个新节点扩展 *****\n"
            << endl;
        for (i = E.s + 1; i < n; i++)
        { // 广度优先搜索, 进行子节点的扩展
            MinHeapNode N;
            if (adjMatrix[E.x[E.s]][E.x[i]] != NoEdge) // E.x[E.s] 是当前要扩展
的父节点, E.x[i] 是被遍历的子节点
            {
                // 可行儿子节点
                cc = E.cc + adjMatrix[E.x[E.s]][E.x[i]]; // 当前费用 = 之前费用 +
新增费用

                lcost = E.lcost - MinOut[E.x[E.s]]; // 更新最小出边费用和
                lb = cc + lcost; // 下界(限界函数)
                if (lb < bestC || bestC == NoEdge) // 子树可能含最优解
                {
                    N.s = E.s + 1; // 进入下一层

```



```

        N.cc = cc;
        N.rcost = lb;
        N.lcost = lcost;
        N.x = new int[n];
        for (j = 0; j < n; j++)
            N.x[j] = E.x[j];
        // 获得新的路径 【换位】
        N.x[E.s + 1] = E.x[i];
        N.x[i] = E.x[E.s + 1];
        pq.push(N); // 加入优先队列
        N.printNode(pq);
    }
}
delete[] E.x; //完成节点扩展
}
if (pq.empty()) // 堆已空
    break;
E = pq.top(); // 取下一扩展节点
pq.pop();
}

if (bestC == NoEdge) // 无回路
    return NoEdge;
for (i = 0; i < n; i++) // 将最优解复制到 v[1:n]
    v[i + 1] = E.x[i];
while (pq.size()) // 释放最小堆中所有节点
{
    E = pq.top();
    pq.pop();
    delete[] E.x;
}
return bestC;
}

int main()
{
    input();
    int res = BBTSP();
    printTravel(res);
    system("PAUSE");
    return 0;
}

```

4. 程序验证，要求有不同的输入、输出，并截图。

E:\算法设计与分析\6\实验6-2: 旅行商问题(分支限界法).exe

```
4
6
1 2 30
1 3 6
1 4 4
2 3 5
2 4 10
3 4 20

===== Node: B =====
最小出边和(lcost): 18    子树费用的下界(lcost): 268501009    当前费用(cc): 0    节点所在层(s): 0
当前解是(x): 1-2-3-4
(优先级队列为空)
-- 当前最优值 (bestC): -1

***** 开始一个新节点扩展 *****

===== Node: C =====
最小出边和(lcost): 14    子树费用的下界(lcost): 44    当前费用(cc): 30    节点所在层(s): 1
当前解是(x): 1-2-3-4
-- 当前优先队列: C(44)--- 当前最优值 (bestC): -1
===== Node: D =====
最小出边和(lcost): 14    子树费用的下界(lcost): 20    当前费用(cc): 6    节点所在层(s): 1
当前解是(x): 1-3-2-4
-- 当前优先队列: D(20)--- 当前最优值 (bestC): -1
===== Node: E =====
最小出边和(lcost): 14    子树费用的下界(lcost): 18    当前费用(cc): 4    节点所在层(s): 1
当前解是(x): 1-4-3-2
-- 当前优先队列: E(18)-D(20)--- 当前最优值 (bestC): -1

***** 开始一个新节点扩展 *****

===== Node: F =====
最小出边和(lcost): 10    子树费用的下界(lcost): 34    当前费用(cc): 24    节点所在层(s): 2
当前解是(x): 1-4-3-2
-- 当前优先队列: D(20)-F(34)--- 当前最优值 (bestC): -1
===== Node: G =====
最小出边和(lcost): 10    子树费用的下界(lcost): 24    当前费用(cc): 14    节点所在层(s): 2
当前解是(x): 1-4-2-3
-- 当前优先队列: D(20)-G(24)--- 当前最优值 (bestC): -1

***** 开始一个新节点扩展 *****

===== Node: H =====
最小出边和(lcost): 9    子树费用的下界(lcost): 20    当前费用(cc): 11    节点所在层(s): 2
当前解是(x): 1-3-2-4
-- 当前优先队列: H(20)-G(24)--- 当前最优值 (bestC): -1
===== Node: I =====
最小出边和(lcost): 9    子树费用的下界(lcost): 35    当前费用(cc): 26    节点所在层(s): 2
当前解是(x): 1-3-4-2
```

5. 算法分析，时间复杂度。

本算法使用了分支界限法来对城市进行选择，测试的例子中，城市数为 4，给出相对应的矩阵，本算法使用了优先队列，其时间复杂度为 $O(n!)$ 。另外，代码中使用了队列（最小堆）来表示结点的优先级，使用一个二维的 n 阶方阵来记录与表示城市之间的距离。所以空间花费为 $O(n*n)$ 。

时间复杂度: $O(n!)$

空间复杂度: $O(n*n)$

结果分析和总结：

这次的实验中用到了队列、树、指针等数据结构，代码比较复杂。在 6-1 中，除了没有填空的地方也有部分代码需要更改，在 6-1 代码中用到了模板函数，模板函数内部是不能嵌套名称相同的模板函数的，所以需要更改模板函数的名称。在调用 EnQueue 函数中，<Type>这个没有删除，导致代码出错。

这次比较难，还得继续向好好学习。

本 次 上 机 成 绩	项目	操作	报告书写	出勤和课堂纪律	其他
	得分				
	成绩合计：				

评语：

教师签字：

日期：